

Paxos 和 Raft 的相似之处以及如何移植优化

王兆国^{†‡}, 赵长庚*, 穆帅[◊], 陈海波^{†‡}, 李金阳* [†]上海交通大学上海市可扩展计算与系统重点实验室[‡]上海交通大学并行与分布式系统研究所*纽约大学计算机科学系[◊]石溪大学计算机科学系

抽象的

近年来, Raft 已经超越 Paxos 成为业界更流行的共识协议。虽然许多研究人员已经观察到这两种协议之间的相似之处, 但没有人表明 Raft 和 Paxos 之间如何正式相关。在本文中, 我们提出了 Raft 和 Paxos 之间的正式映射, 并利用这些知识将某一类优化从 Paxos 移植到 Raft。特别是, 我们的移植方法可以自动生成优化的协议规范, 并保证正确性。作为案例研究, 我们将 Mencius 和 Paxos Quorum Lease 两种优化移植并评估到 Raft。

关键词

Paxos、Raft、优化移植

ACM参考格式:

王兆国, 赵长庚, 穆帅, 陈海波, 李金阳. 2019. 关于 Paxos 和 Raft 之间的相似之处, 以及如何移植优化. 2019 年 ACM 分布式计算原理研讨会 (PODC' 19), 2019 年 7 月 29 日至 8 月 2 日, 加拿大安大略省多伦多. ACM, 美国纽约州纽约市, 10 页. <https://doi.org/10.1145/3293611.3331595>

1 简介

共识协议使服务器能够在某些服务器发生故障和任意网络延迟的情况下就执行操作的顺序达成一致。Classic Paxos [18] 是最古老、研究最深入的共识协议之一。然而, 近年来, Raft [31] 逐渐取代 Paxos 成为共识协议的选择, 尤其是在分布式计算中。在行业中。许多研究人员观察到 Raft 和 Paxos 具有某些相似之处。然而, 没有人表明这两个协议在正式意义上是如何相关的。事实上, 这种正式关系存在吗?

虽然这看起来像是一种迂腐的努力, 但研究 Raft 和 Paxos 之间的正式映射是有意义的, 原因有两个。首先, 将 Raft 和 Paxos 联系起来有助于加深我们对这两种协议的理解。特别是, 它使我们能够

阐明哪些设计决策使 Raft 比 Paxos 更容易理解或更高效。其次, Paxos 不是一个孤立的协议, 而是由近二十年研究的一大系列变体和优化组成 [16,7,25,22,29,28]。这些 Paxos 变体的范围包括减少广域操作的延迟、平衡副本负载、针对大多数无冲突的工作负载进行优化, 以及容忍拜占庭错误。了解 Raft 与 Paxos 的关系允许我们将其中一些优化移植到 Raft, 而无需重新发明轮子。

在本文中, 我们尝试使用细化映射在 Raft 和 Paxos 之间建立正式的连接。我们表明, 除了两个协议之间的大致相似之处之外, Raft 与 Paxos 在几个微妙的细节上有所不同, 例如如果追随者的日志比领导者长, 则允许追随者删除额外的条目。不幸的是, 这两个协议之间的这些差异阻碍了它们之间的直接细化映射。因此, 我们制作了 Raft 的一个变体, 称为 Raft*, 它是通过消除这些表面差异而对 Paxos 进行的改进。

我们使用 Raft* 和 Paxos 之间的细化映射将 Paxos 文献中的现有思想移植到 Raft 的世界。具体来说, 我们开发了一种自动移植方法, 能够将某一类 Paxos 优化移植到 Raft*。可以自动移植的特定优化类别是那些不会改变 Paxos 中原始状态的优化。对于这些优化, 我们推导了一组规则, 将它们应用于 Raft*, 使得生成的协议自动细化 Raft* 和 Paxos 优化, 从而保证正确性。由于 Raft* 与 Raft 非常相似, 因此衍生协议包含了 Raft 的所有属性, 并通过 Paxos 优化进行了改进。作为案例研究, 我们选择了两个已发表的 Paxos 优化: Mencius [25] 和 Paxos Quorum Lease [28], 它们分别在负载均衡和延迟方面改进了 Paxos 的一个或多个方面。我们已将这两个协议移植到 Raft*。

我们评估了 Amazon AWS 上 Raft* 优化的性能优势, 其中数据被复制到多个地理位置不同的数据中心。对于每个优化, 我们都表明 Raft 变体与文献中的 Paxos 变体具有相似的优点。

总而言之, 我们做出以下贡献:

- 我们通过展示 Paxos 和 Raft* (Raft 的紧密变体) 之间的细化映射来揭示 Raft 和 Paxos 之间的形式关系 (第 3 节)。
- 我们定义了跨协议的移植优化问题, 并开发了一种自动移植协议的方法。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '19, July 29-August 2, 2019, Toronto, ON, Canada

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6217-7/19/07...\$15.00

<https://doi.org/10.1145/3293611.3331595>

从一种协议到另一种协议的受限优化类别（第4节）

- 我们将 Mencius 和 Quorum Lease 从 Paxos 移植到 Raft，并提供 Raft 协议优化版本的实验评估（第6节）

局限性。本文代表了将优化从一个协议系列自动移植到另一个相关协议系列的第一次尝试。我们的方法有一些限制，限制了它的适用性。首先，为了端口优化，我们必须提供两个相关协议之间的细化映射。因此，我们只能自动将优化从 Paxos 移植到 Raft*，而不是直接移植到 Raft。其次，只能自动移植特定受限类别的优化。具体来说，此类优化不得改变协议状态（第4.2节）。最后，优化移植是在协议规范级别而不是实现级别完成的。程序员仍然需要手动实现派生协议。

2 概述

2.1 背景

Paxos [18]。Paxos 使用自下而上的方法解决共识问题。首先，开发单一法令 Paxos 是为了让服务器就单一值达成一致。然后，MultiPaxos 建立在单法令 Paxos 的基础上，以就一系列操作达成一致。

在单法令 Paxos 中，服务器通过两阶段协议达成共识。在第一阶段，服务器选择一个全局唯一的提案编号（称为选票）并向每个服务器发送一个准备 RPC。如果接收到准备回复的服务器没有看到更高的选票，则它会成功。在其回复中，服务器包含它曾经接受过的最高选票，如果它从未接受过任何选票，则为空。如果提议服务器至少收到大多数成功回复，则它进入第二阶段，否则，它会以更高的选票重试。

在第2阶段，服务器选择其选票的值并将其通过 Accept RPC 发送到每个服务器。如果第一阶段回复不包含任何先前接受的值，则这可以是任何值（通常是服务器想要启动的操作）；否则，它必须是回复中投票数最高的值（由其他服务器发起的操作）。接收 Accept RPC 的服务器将接受该值，如果没有看到更高的选票，则会回复成功。如果提议服务器收到大多数成功回复，则它认为该值已被选择（或同等地，已提交）。它可以立即通知其他服务器已提交的操作，或者通过后续通信搭载此信息。

MultiPaxos 建立在单一法令 Paxos 的基础上，以就一系列操作达成一致。特别是，MultiPaxos 尝试使用单独的 Paxos 实例就序列中每个位置的操作达成一致。MultiPaxos 还通过允许并发实例并对这些实例的第一阶段消息进行批处理来优化性能。图1给出了MultiPaxos的伪代码。在本文的其余部分中，Paxos 指的是 Paxos 的多法令版本，而单法令 Paxos 指的是单法令版本。

筏[31]。与 Paxos 自下而上的方法不同，Raft 以自上而下的方式解决共识问题，通过跨服务器复制操作日志，而不分解为单法令共识。Raft 协议由两部分组成：选举领导者和由选举的领导者复制日志条目。每个服务器

维持单调递增的项数。对于领导者选举，候选服务器会增加其任期号并向所有服务器发送 RequestVote RPC 来为自己收集选票以成为领导者。选举按相应的任期号排序，如果节点已经处理了来自不同候选人的更高任期或相同任期的请求，则节点会拒绝 RequestVote。Raft 还对领导者选举添加了另一个限制：如果节点的日志比发送者的日志更新，则节点会拒绝 RequestVote。如果候选人在其 RequestVote RPC 上获得了多数法定人数的成功投票，则该候选人将成为该任期的当选领导者。

当选的领导者会批量处理客户端操作，并使用 AppendEntries RPC 将它们复制到所有其他服务器（称为追随者）。如果服务器看到比发送者更高的术语，则它会拒绝 AppendEntries 请求。AppendEntries RPC 还允许接收方将其日志与发送方同步：如果接收方丢失条目，则接收方会赶上，并删除发送方日志中未找到的无关条目。如果大多数法定服务器成功回复，则领导者认为 AppendEntries 中的操作已提交。图2展示了Raft的伪代码（忽略蓝色代码）。

2.2 我们的方法

乍一看，Raft 与 Paxos 有很多相似之处。两种协议都有两个阶段。Raft的RequestVote对应Paxos中的Phase1a。如果收到大多数 ok 回复，则 RequestVote 和 Phase1a 均被视为成功。之后，Raft 使用 AppendEntries 将操作复制到其他服务器，类似于 Paxos 如何使用 Phase2a 传播与特定单法令实例相关的操作。在这两种协议中，如果服务器看到更高的术语/选票，则拒绝 AppendEntries/Accept 请求，并且仅当大多数服务器返回 ok 时才认为该操作已提交。

我们的工作使用细化映射 [1] 来正式捕获 Paxos 和 Raft 之间的联系。细化映射通常用于证明较低级别的规范正确地实现了较高级别的规范。例如，现有的工作[36]已将Paxos、Viewstamp Replication和ZAB映射到高级抽象共识算法，而不是直接将它们相互关联。相比之下，为了阐明 Paxos 和 Raft 之间的关系，我们尝试直接提供两个协议之间的细化映射。

事实证明，存在一些细微的差异，使得 Raft 无法直接细化映射到 Paxos。尽管如此，我们可以稍微修改 Raft 以消除这些表面差异并创建一个相近的变体协议（称为 Raft*）。通过展示从 Raft* 到 Paxos 的细化映射，我们阐明了 Raft 和 Paxos 之间的联系。

除了提供对协议更深入的理解之外，Raft* 和 Paxos 之间的细化映射是否可以用于某些实际目的？以此问题为指导，我们继续探索如何将 Paxos 的优化迁移到 Raft。基于细化映射，我们开发了一种将某些 Paxos 优化移植到 Raft* 的方法，从而保证最终协议的正确性。具体来说，一些 Paxos 优化只读取但不改变 Paxos 的状态变量。直觉是根据细化映射，通过用 Raft* 变量替换 Paxos 变量来移植这些优化。例如，

1	function Phase1a(s):	26	function Phase2a(s, i, v):	26
2	s.ballot = s.ballot + 1	27	if s.phase1Succeeded	27
3	unchosen = smallest unchosen instance id	28	&& (s.instances[i].val == v	28
4	s sends <'prepare', s.ballot, unchosen> to all	29	s.instances[i] == Empty)	29
5		30	then	30
6	function Phase1b(s):	31	send <'accept', i, v, s.ballot> to all	31
7	if s receives <'prepare', b, unchosen>	32		32
8	&& b > s.ballot	33	function Phase2b(s):	33
9	then	34	if s receives <'accept', i, v, b> && b ≥ s.ballot	34
10	s.ballot = b	35	then	35
11	s.phase1Succeeded = false	36	if b > s.ballot	36
12	reply <'prepareOK', s.ballot,	37	then	37
13	instances with id ≥ unchosen>	38	s.phase1Succeeded = false	38
14		39	s.ballot = b	39
15	function Phase1Succeed(s):	40	s.instances[i].bal = b	40
16	if s receives <'prepareOK', b, instances>	41	s.instances[i].val = v	41
17	from f + 1 acceptors with the same b	42	reply <'acceptOK', i, v, b>	42
18	&& b == s.ballot	43		43
19	then	44	function Learn(s):	44
20	start = smallest unchosen instance id	45	if s receives same <'acceptOK', i, v, b>	45
21	end = largest id of received instances	46	from f + 1 acceptors	46
22	for i in start ... end	47	then	47
23	s.instances[i] = safeEntry(received	48	s.instances[i].val = v	48
24	instances with id i)	49	s.instances[i].bal = b	49
25	s.phase1Succeeded = true	50	add s.instances[i] to s.chosenSet	50

(a) Phase 1

(b) Phase 2

图 1: MultiPaxos

考虑 Paxos 的附加检查点机制，它可以保存系统状态和最后应用的实例 ID。根据细化映射，实例 id 被映射到日志索引。因此，当将检查点机制移植到 Raft* 时，我们可以用日志中最后应用的索引替换最后应用的实例 id，并保证正确性。

3 将 RAFT 连接到 PAXOS

细化映射。我们使用细化映射来描述两个协议之间的等价性。直观上，为了表明协议 $\mathcal{B}$ 改进了协议 $\mathcal{A}$ ，我们必须表明 $\mathcal{B}$ 状态空间中的每个状态如何映射到 $\mathcal{A}$ 状态空间中的某个状态，使得 $\mathcal{B}$ 中的任何状态转换序列在映射下对应于 $\mathcal{A}$ 中的有效状态转换序列。细化映射的直观定义足以满足本节的需要。在后面的部分（第 4 节）中，我们提出了用于优化移植的细化映射的更正式的定义。

为什么 Raft 不能直接映射到 Paxos。理想情况下，我们应该能够直接将 Raft 改进为 Paxos。不幸的是，由于两个原因无法做到这一点。首先，Raft 强制所有接受 Leader 的 AppendEntries 的服务器与 Leader 的日志相匹配。因此，如果追随者的日志比领导者的日志长，追随者将删除多余的条目。当映射到 MultiPaxos 时，这样的“擦除”步骤将对应于服务器删除某个 Paxos 实例先前接受的值，这是 Paxos 中永远不会发生的状态转换。Raft 通过按顺序提交日志索引来确保擦除步骤的安全，以便其第一阶段交换确保领导者的日志包含所有可能提交的条目。相比之下，Paxos 会无序地提交实例。因此，Paxos 的提议服务器必须从其他服务器获取不同未提交实例的安全值，并且永远不会删除（但仅覆盖）其他服务器上接受的值。二、每个日志条目中的术语编号

Raft 中无法映射到 Paxos 中每个实例接受的选票号。原因是 Raft 的领导者从不修改其现有的日志条目。因此，新当选的领导者在其任期 t 时将复制先前未提交的日志条目，任期 $t' < t$ 不做任何更改。这种行为在 Paxos 中没有对应的行为。Paxos 中的提议服务器总是用当前的选票号覆盖已接受实例的选票号。不更改日志条目的术语编号会产生微妙的正确性影响，因此 Raft 论文（[31] 第 5.4.2 节）必须添加额外的规则来防止提交值丢失。

筏*。我们稍微修改 Raft 以创建一个接近的变体，称为 Raft*，其中存在到 Paxos 的细化映射。图 2 以伪代码形式展示了 Raft* 的规范（包括蓝色代码）。Raft* 与 Raft 相同，除了两个引入的差异之外，基于 Raft 无法改进 Paxos 的两个原因。首先，当对候选人的 RequestVote 做出“确定”响应时，服务器会包含在其回复中包含候选人日志中不存在的所有额外条目（第 68-69 行）。领导者在多数法定回复中选择安全值来扩展其日志（第 77-81 行）。如果接受者的日志比领导者的长，则接受者会拒绝领导者的追加请求（第 110 行）。其次，每个条目都添加一个投票字段。追加新条目时，Raft* 会将所有条目的选票更改为新条目的任期（第 99-100 行）。

使用 Raft* 精炼 Paxos。图 3 给出了 Raft* 和 Paxos 之间的细化映射。其中大多数都很简单。例如，索引不大于 commitIndex（已提交条目）的 Raft 条目将映射到 chosenSet（选择实例）中的 Paxos 实例。我们仅解释映射的细微差别。首先，Raft* 中的大多数消息与 Paxos 中的对应消息的内容不同。例如，requestVote 包括 lastIndex 和 lastTerm，而不是未选择实例的最小 id。这是因为，如果两个日志具有相同的条目

<pre> 52 // s starts an election for the new leader 53 function RequestVote(s): 54 s.currentTerm = s.currentTerm + 1 55 s.sends <'requestVote', s.currentTerm, s.lastIndex, 56 s.log[s.lastIndex].term> to all 57 58 // s receives leader election request 59 function ReceiveVote(s): 60 if s receives <'requestVote', term, lastIndex, lastTerm> 61 && term > s.currentTerm 62 && s.log[lastIndex].term < lastTerm 63 (s.log[lastIndex].term = lastTerm 64 && s.lastIndex ≥ lastIndex) 65 then 66 s.currentTerm = term 67 s.isLeader = false 68 extraEnts = non-empty entries in s.log after lastIndex 69 reply <'requestVoteOK', s.currentTerm, extraEnts> 70 71 // s becomes the new leader in the vote phase 72 function BecomeLeader(s): 73 if s receives <'requestVoteOK', term, ents> from f + 1 74 acceptors with same term && term == s.currentTerm 75 then 76 max = largest index of received entries 77 for i in s.lastIndex + 1 ... max 78 e = safeEntry(received entries of index i) 79 s.log[i].bal = s.currentTerm 80 s.log[i].term = s.currentTerm 81 s.log[i].val = e.val 82 s.isLeader = true 83 s.lastIndex = max 84 85 86 87 88 89 90 </pre>	<pre> 91 // s appends vals to the log from i and 92 // broadcasts all entries after prev to replicas 93 function AppendEntries(s, i, vals, prev): 94 if s.isLeader && i == s.lastIndex + 1 then 95 for each v in vals 96 s.log[s.lastIndex+1].val = v 97 s.log[s.lastIndex+1].term = s.currentTerm 98 s.lastIndex = s.lastIndex + 1 99 for i in prev + 1 ... lastIndex 100 s.log[i].bal = s.currentTerm 101 ents = s.log entries after prev 102 pTerm = s.log[prev].term 103 send <'append', s.currentTerm, prev, pTerm, ents, s.commitIndex> to all 104 105 // s receives append request from the leader 106 function ReceiveAppend(s): 107 if s receives <'append', term, prev, pTerm, ents, commit> 108 && term ≥ s.currentTerm 109 && s.log[prev].term == pTerm 110 && s.lastIndex ≤ prev + length(ents) 111 then 112 if term > s.currentTerm then 113 s.isLeader = false 114 s.currentTerm = term 115 s.lastIndex = prev + length(ents) 116 s.commitIndex = max(commit, s.commitIndex) 117 replace entries after s.log[prev] with ents 118 reply <'appendOK', s.currentTerm, s.lastIndex> 119 120 // the leader s learns the committed entries 121 function LeaderLearn(s): 122 if s receives <'appendOK', term, index> from f acceptors with 123 the same term 124 && s.isLeader 125 && term == s.currentTerm 126 then 127 minIndex = minimal received index 128 s.commitIndex = max(s.commitIndex, minIndex) 129 </pre>
--	---

(a) Phase 1

(b) Phase 2

图 2: Raft*。蓝色代码是Raft*引入的

如果给定索引处的术语，则这两个日志在给定索引之前是相同的（日志匹配属性）。此外，Raft 能够使用最后一个条目的术语 (lastTerm) 来检测日志中的每个条目是否都是最新的。其次，Raft*的leader直接在AppendEntries和BecomeLeader中将条目追加到日志中。这可以被认为是在隐式地向自身发送一个追加，然后从自身接收一个appendOk。显式和隐式的append/appendOk都可以暗示accept/acceptOk。最后，Raft*的函数可能意味着Paxos中的多个函数。例如，AppendEntries意味着Phase2a和Phase2b：领导者首先隐式接受该条目，然后将其发送给其他人。细化映射的正式规范和证明可以在[37]中找到。

4 一种移植优化方法

在本节中，我们将展示如何利用细化映射将某些Paxos优化自动移植到Raft*。首先，我们正式定义优化移植问题（第4.1节）。然后我们精确地描述哪一类优化适合我们的方法（第4.2节）。最后，我们描述了移植优化的算法，以确保派生的协议规范是正确的（第4.3节）。

4.1 问题定义

指定协议。为了自动跨协议移植一些优化，我们必须以正式的方式描述协议及其优化。我们将协议指定为状态机，可以通过其初始状态和一组允许的状态转换来定义。我们使用TLA⁺语言[19]来指定状态机。

在TLA⁺中，通过描述称为“下一步”操作的一组允许的状态转换来指定协议，表示为子操作 $a_1 \vee a_2 \vee \dots$ 的集合，其中 $\vee$ 是或（析取）运算符。每个子动作 a_i 是一个带有一个或多个子句的连取形式的公式；这些子句指定状态转换的启用条件和下一个状态值。作为示例，我们考虑支持两个操作 Put(k, v) 和 Get(k) 的键值存储。图4a显示了其TLA⁺规范。键值存储的内部状态是一个哈希表（表），其中每个条目对应一个集合。对于每个潜在的键和值，其下一状态操作 (Next) 由两个子操作 Put(k, v) 和 Get(k) 组成。在图4a中，子操作 Put(k, v) 被定义为等于 $(\hat{=})$ 布尔公式，断言键 k 的哈希表条目必须在下一个状态（第2行）包含值 v。在TLA⁺中，将 ' 符号附加到变量上表示其在新状态下的值。因此，只有当下一个状态中键 k 的哈希表条目等于时，公式 $table[k]=\{v\}$ 才为真

		Raft*	MultiPaxos
variables	Constant	Quorums	Quorums
	per server	currentTerm isLeader entries with index $\leq$ commitIndex	ballot phase1succeeded chosenSet
	per instance	entry.index entry.val entry.bal	instance.id instance.val instance.bal
	messages	requestVote requestVoteOK (im/ex) append (im/ex) appendOK	prepare prepareOK accept acceptOK
functions		RequestVote	Phase1a
		ReceiveVote	Phase1b
		BecomeLeader	Phase1Succeed Phase2a Phase2b
		AppendEntries	Phase2a Phase2b
		ReceiveAppend	Phase2b
		LeaderLearn	Learn

图 3: Raft* 和 MultiPaxos 之间的映射。“im”代表隐式。“ex”代表显式

v. 我们使用输出变量来表示返回给用户的值，因此 $\text{Get}(k)$ 使用子句 $\text{output}' = \text{table}[k]$ 来断言新状态下输出变量的值。在 $\mathcal{A}$ 的这个示例中，不存在涉及多个连接形式子句的子操作，因为 Put 或 Get 都没有启用条件。我们将在后续示例中看到更复杂的子操作。

使用 TLA^+ 进行细化映射。使用 TLA^+ 规范，我们可以正式描述细化映射。我们使用 $\mathcal{B} \Rightarrow \mathcal{A}$ 来表示 $\mathcal{B}$ 具有到 $\mathcal{A}$ 的细化映射。（例如， $\text{Raft}^* \Rightarrow \text{Paxos}$ ）。设 $\text{Var}_{\mathcal{A}}$ （或 $\text{Var}_{\mathcal{B}}$ ）表示 $\mathcal{A}$ （或 $\mathcal{B}$ ）的状态变量。为了证明 $\mathcal{B} \Rightarrow \mathcal{A}$ ，我们需要找到一些函数 f 将 $\mathcal{B}$ 的状态空间映射到 $\mathcal{A}$ 的状态空间，即 $\text{Var}_{\mathcal{A}} = f(\text{Var}_{\mathcal{B}})$ 。假设 a_i 是 $\mathcal{A}$ 中的某个子动作，当我们用 $\text{Var}_{\mathcal{A}} = f(\text{Var}_{\mathcal{B}})$ 替换 a_i 中的 $\text{Var}_{\mathcal{A}}$ 时，我们使用术语 a_i 来指代合取公式。如果 $\mathcal{B}$ 在 f 下精炼 $\mathcal{A}$ ，则 $\mathcal{B}$ 中的每个子操作 b_i 隐含 $\mathcal{A}$ 中的某个子操作 a_j 或无操作步骤 1，即 $b_i \Rightarrow \overline{a_j} \vee f(\text{Var}_{\mathcal{B}}') = f(\text{Var}_{\mathcal{B}})$ ，其中 $\Rightarrow$ 是隐含的布尔运算符。

图 4b 显示了在日志中存储数据的示例协议 $\mathcal{B}$ 。子操作 $\text{Write}(i, v)$ 在索引 i 处的日志末尾存储一个值。第 2 行的连接子句确保值连续存储在日志中的不变性。子操作 $\text{Read}(i)$ 读取日志条目 i 。图 4b 中的协议 $\mathcal{B}$ 在状态映射下细化了图 4a 中的协议 $\mathcal{A}$ ，该状态映射将日志的第 i 个条目映射到具有密钥 $k = i$ 的哈希表条目。 $\mathcal{B}$ 中的子操作 Write 意味着 Put in $\mathcal{A}$ ，而 Read 意味着 Get。 $\mathcal{A}$ 和 $\mathcal{B}$ 之间的细化的形式证明可以在[37]中找到。

定义移植优化问题。非正式地，给定两个协议 $\mathcal{A}$ 和 $\mathcal{B}$ ， $\mathcal{B} \Rightarrow \mathcal{A}$ ，我们希望对协议 $\mathcal{A}$ 进行一些现有的优化，以使其也适用于协议 $\mathcal{B}$ 。更重要的是，我们需要适应遵循算法

(而不是手动)程序，可以保证所得协议的正确性。

为了正式地陈述任务，我们得到了一些协议 $\mathcal{A}$ 、其优化版本 $\mathcal{A}^\Delta$ 和另一个细化 $\mathcal{A}$ 的协议 $\mathcal{B}$ ，所有这些都在 TLA^+ 中指定。此外，我们假设所有三个协议 $\mathcal{A}$ 、 $\mathcal{A}^\Delta$ 、 $\mathcal{B}$ 都已被证明是正确的。移植优化的问题是自动导出协议 $\mathcal{B}^\Delta$ 的 TLA^+ 规范，使得 $\mathcal{B}^\Delta$ 提高 $\mathcal{B}$ 的性能并保证正确。我们注意到，目标是导出优化协议的 TLA^+ 规范，而不是自动生成优化协议的实现。

如上所述的优化移植问题是相当普遍的。特别是，我们不对协议给出的正确性证明的类型做出任何假设，也不对 $\mathcal{A}$ 和 $\mathcal{A}^\Delta$ 之间的任何正式关系做出任何假设。然而，为了使问题可跟踪，我们设计了一种仅适用于受限优化类别的解决方案，如下所述。

4.2 非变异优化

给定协议 $\mathcal{A}$ 及其优化变体 $\mathcal{A}^\Delta$ ，我们认为优化是 $\mathcal{A}^\Delta$ 和 $\mathcal{A}$ 规范之间的差异。特别地， $\mathcal{A}^\Delta$ 的状态变量包括 $\mathcal{A}$ 的所有状态变量并且可以包含由优化引入的附加变量。 $\mathcal{A}^\Delta$ 的每个子操作可以采用三种形式：

- 添加的子操作。这是一个与 $\mathcal{A}$ 中现有子操作没有关系的子操作。
- 未更改的子操作。该子操作与 $\mathcal{A}$ 中的现有子操作相同。
- 修改后的子操作。这是通过添加额外的连接子句从 $\mathcal{A}$ 中的现有子操作派生的子操作。2

我们提出的移植优化方法适用于有限类别的优化，我们将其称为非变异优化。为了使优化 $\mathcal{A}^\Delta$ 被视为非变异，我们要求其添加的子操作及其修改的子操作中添加的子句都不会改变 $\mathcal{A}(\text{Var}_{\mathcal{A}})$ 的状态变量。子操作可以自由改变 $\mathcal{A}^\Delta$ 添加的新状态变量 (Var_{Δ})。

图 4c 显示了协议 $\mathcal{A}^\Delta$ ，作为键值存储协议 $\mathcal{A}$ 的非变异优化示例。优化后的协议 $\mathcal{A}^\Delta$ 添加了一个新的状态变量大小，用于跟踪哈希表中存储了多少值。比较图 4c 和图 4a，我们可以看到 $\mathcal{A}^\Delta$ 将新子句（第 4 行）添加到现有子操作 Put 中，并且没有新子操作。由于 new 子句不会修改 $\mathcal{A}$ 的状态（表），因此 $\mathcal{A}^\Delta$ 是一种非变异优化。

非变异优化不仅允许我们使用 4.3 节中的方法将优化从 $\mathcal{A}$ 移植到 $\mathcal{B}$ ，它还有一个额外的好处，即 $\mathcal{A}^\Delta$ 可以在忽略额外状态的相同状态映射函数下优化 $\mathcal{A}$ 。因此，非变异优化总是保证正确性。相比之下，状态突变优化可能有也可能没有到 $\mathcal{A}$ 的细化映射，因此其正确性需要单独的证明。

¹The no-op step is commonly called a stuttering step [1, 23]

1: VARIABLES table, output 2: Put(k, v) $\triangleq$ table'[k] = {v} 3: Get(k) $\triangleq$ output' = table[k] 4: Init $\triangleq \forall k \in \text{Nat}: \text{table}[k] = \{\}$ 5: Next $\triangleq \exists k \in \text{Nat}, v \in \text{Values}: \text{Put}(k, v) \vee \text{Get}(k)$	1: VARIABLES logs, output 2: Write(i, v) $\triangleq (i = 0 \vee \text{logs}[i-1] \neq \{\})$ 3: $\wedge \text{logs}'[i] = v$ 4: Read(i) $\triangleq$ output' = logs[i] 5: Init $\triangleq \forall i \in \text{Nat}: \text{logs}[i] = \{\}$ 6: Next $\triangleq \exists i \in \text{Nat}, v \in \text{Values}: \text{Write}(i, v) \vee \text{Read}(i)$
(a) A key-value store ($\mathcal{A}$).	(b) The protocol $\mathcal{B}$ that stores data in a log. $\mathcal{B}$ refines the key-value store $\mathcal{A}$.
1: VARIABLES table, output, size 2: Put(k, v) $\triangleq$ table[k] = {v} 3: $\wedge \text{table}'[k] = \{v\}$ 4: $\wedge \text{size}' = \text{size} + 1$ 5: Get(k) $\triangleq$ output' = table[k] 6: Init $\triangleq \forall k \in \text{Nat}: \text{table}[k] = \{\} \wedge \text{size} = 0$ 7: Next $\triangleq \exists k \in \text{Nat}, v \in \text{Values}: \text{Put}(k, v) \vee \text{Get}(k)$	1: VARIABLES logs, output, size 2: Write(i, v) $\triangleq$ logs[i] = {v} 3: $\wedge (i = 0 \vee \text{logs}[i-1] \neq \{\})$ 4: $\wedge \text{logs}'[i] = \{v\}$ 5: $\wedge \text{size}' = \text{size} + 1$ 6: Read(i) $\triangleq$ output' = logs[i] 7: Init $\triangleq \forall k \in \text{Nat}: \text{logs}[i] = \{\} \wedge \text{size} = 0$ 8: Next $\triangleq \exists i \in \text{Nat}, v \in \text{Values}: \text{Write}(i, v) \vee \text{Read}(i)$
(c) The optimized protocol $\mathcal{A}^\Delta$.	(d) The generated TLA ⁺ specification of $\mathcal{B}^\Delta$.

图 4: 示例的 TLA⁺ 规范。

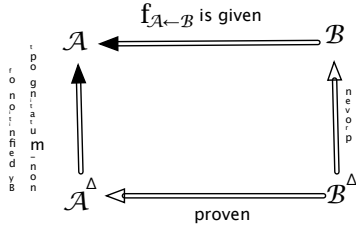


图 5: 给定协议之间的细化映射。 $\mathcal{A}^\Delta$ 是使用非变异优化的 $\mathcal{A}$ 的优化版本。 $\mathcal{B}^\Delta$ 是我们在 4.3 节中的方法生成的 $\mathcal{B}$ 的优化版本。

4.3 如何移植非变异优化

我们只考虑移植非变异优化的情况。此外，如果优化读取协议 $\mathcal{A}$ 的参数，我们的方法还需要从 $\mathcal{B}$ 到 $\mathcal{A}$ 的参数映射。

参数映射。令 $P_{\mathcal{A}}$ 和 $P_{\mathcal{B}}$ 分别为 $\mathcal{A}$ 和 $\mathcal{B}$ 的参数变量。如果存在一个函数 farg_{Δ} 将 $\mathcal{B}$ 中子动作的参数映射到 $\mathcal{A}$ 中子动作的参数，我们就说 $\mathcal{B}$ 有参数映射到 $\mathcal{A}$ ，即 $P_{\mathcal{A}} = \text{farg}_{\Delta}(P_{\mathcal{B}})$ 。³ 在 $\mathcal{A}^\Delta$ 中修改的子动作中添加的额外子句可以使用参数变量。因此，需要参数映射才能正确翻译要在 $\mathcal{B}^\Delta$ 中的相应子操作中使用的这些子句。

移植优化。我们现在准备好描述如何转换 $\mathcal{A}^\Delta$ 的规范来创建 $\mathcal{B}^\Delta$ ，从而将优化从 $\mathcal{A}$ 移植到 $\mathcal{B}$ 。首先，我们获取 $\mathcal{B}^\Delta$ 的状态变量

²If the derivation deletes an existing conjunctive clause, then the resulting subaction of $\mathcal{A}^\Delta$ must be viewed as an added subaction instead of a modified one.

³To put it more formally, given parameter mapping $P_{\mathcal{A}} = \text{farg}_{\Delta}(P_{\mathcal{B}})$, we use $\text{Next}_{\mathcal{A}}$ to refer to the formula after substituting state variables $\text{Var}_{\mathcal{A}}$ with $f(\text{Var}_{\mathcal{B}})$ and parameter variables $P_{\mathcal{A}}$ with $\text{farg}_{\Delta}(P_{\mathcal{B}})$. farg_{Δ} is a valid parameter mapping if $\text{Next}_{\mathcal{B}} \Rightarrow \text{Next}_{\mathcal{A}}$.

作为 $\text{Var}_{\mathcal{B}^\Delta} = \text{Var}_{\mathcal{B}} \cup \text{Var}_{\Delta}$ 。我们还通过使用状态映射 $\text{Var}_{\mathcal{A}} = f(\text{Var}_{\mathcal{B}})$ 替换 $\mathcal{A}$ 的每个状态变量，从 $\text{Init}_{\mathcal{B}}$ 和 $\text{Init}_{\mathcal{A}^\Delta}$ 获得 $\mathcal{B}^\Delta$ 的初始状态 ($\text{Init}_{\mathcal{B}^\Delta}$)。接下来，我们从 $\mathcal{A}^\Delta$ 的每个子动作 a^Δ_i 和无操作步骤生成 $\mathcal{B}^\Delta$ 的子动作。分三种情况：

Case-1: a^Δ_i 是一个添加的子操作。我们通过用 $f(\text{Var}_{\mathcal{B}})$ 替换状态变量 $\text{Var}_{\mathcal{A}}$ 并保持 Var_{Δ} 不变，将 a^Δ_i 转换为相应的添加子动作 b^Δ_i 。

情况 2: a^Δ_i 是一个未更改的子操作，等于 $\mathcal{A}$ 中的 a_i ，或无操作步骤。 $\mathcal{B}$ 中有一组子动作，根据细化映射 $f_{\mathcal{B} \rightarrow \mathcal{A}}$ 隐含 a_i 。我们直接将子操作集添加到 $\mathcal{B}^\Delta$ 。

Case-3: a^Δ_i 是 $\mathcal{A}$ 中 a_i 的修改子操作。同样， $\mathcal{B}$ 中有一组子动作，根据 $f_{\mathcal{B} \rightarrow \mathcal{A}}$ 暗示 a_i 。假设 b_j 是集合中的子动作。如果 b_j 尚未添加（情况 2），我们将 b_j 添加到 $\mathcal{B}^\Delta$ 。此外，我们通过替换 $\text{Var}_{\mathcal{A}} = f(\text{Var}_{\mathcal{B}})$ 和 $P_{\mathcal{A}} = \text{farg}_{\Delta}(P_{\mathcal{B}})$ 将 a^Δ_i 添加的额外子句包含在 b_j 中。

正确性。我们证明生成的 $\mathcal{B}^\Delta$ 规范是正确的。证明包含两部分。首先，我们需要证明 $\mathcal{B}^\Delta$ 正确地合并了 $\mathcal{A}^\Delta$ 中的优化。这可以通过证明 $\mathcal{B}^\Delta$ 细化 $\mathcal{A}^\Delta$ 来证明，因此 $\mathcal{B}^\Delta$ 保留了优化引入的不变量。其次，我们还需要证明 $\mathcal{B}^\Delta$ 仍然正确。原始协议 $\mathcal{B}$ 。这可以通过证明 $\mathcal{B}^\Delta$ 改进了 $\mathcal{B}$ 来证明，因此 $\mathcal{B}^\Delta$ 保留了原始协议 $\mathcal{B}$ 的不变量。

作为总结，图 5 说明了四种协议 $\mathcal{A}$ 、 $\mathcal{B}$ 、 $\mathcal{A}^\Delta$ 、 $\mathcal{B}^\Delta$ 之间存在的细化映射。接下来，我们提供 $\mathcal{B}^\Delta$ 的细化映射的证明草稿。

首先，我们证明 $\mathcal{B}^\Delta$ 使用状态映射函数细化 $\mathcal{A}^\Delta$ ，该函数根据 f 将 $\mathcal{B}$ 的状态变量映射到 $\mathcal{A}$ 的状态变量，并保持优化 Δ 引入的变量不变。为了证明这种细化映射的正确性，我们必须证明 $\mathcal{B}^\Delta$ 的初始状态意味着 $\text{Init}_{\mathcal{A}^\Delta}$ ，而 $\mathcal{B}^\Delta$ 的下一个状态操作 ($\text{Next}_{\mathcal{B}^\Delta}$) 意味着 $\text{Next}_{\mathcal{A}^\Delta}$ 。前者的含义比较简单，所以我们重点讨论后者。

为了显示 $\text{Next}_{\mathcal{B}^\Delta}$ 隐含 $\text{Next}_{\mathcal{A}^\Delta}$ ，我们表明每个 $\mathcal{B}^\Delta$ 的子操作 (b^{Δ}_i) 隐含 $\mathcal{A}^\Delta$ 的某个子操作或无操作步骤。根据我们的方法，在三种情况之一中， b^{Δ}_i 可以添加到 $\mathcal{B}^\Delta$ 。对于情况 1 和 2，很容易通过构造证明 b^{Δ}_i 意味着 a^{Δ}_i 或无操作步骤。在情况 3 中， b^{Δ}_i 由 b_i 和 $\mathcal{A}^\Delta$ 中的子操作 a^{Δ}_j 构造，使得 $a^{\Delta}_j = a_j \wedge \Delta_{a_j}$ 、 $b^{\Delta}_i = b_i \wedge \Delta_{a_j}$ 。 Δ_{a_j} 是通过优化添加到 a_j 以形成 a^{Δ}_j 的额外连接子句的集合。 Δ_{a_j} 是通过将变量 $\text{Vara} = f_{\mathcal{B} \rightarrow \mathcal{A}}(\text{Varb})$ 和参数 Pa 替换为 $\text{fargs}(\text{Pb})$ 从 Δ_{a_j} 获得的。由于 $b_i \Rightarrow \overline{a_j}$ ，我们有 $b^{\Delta}_i \Rightarrow \overline{a_j} \wedge \Delta_{a_j}$ ，它相当于 $b^{\Delta}_i \Rightarrow \overline{a^{\Delta}_j}$ 。

接下来，我们证明 $\mathcal{B}^\Delta$ 细化了 $\mathcal{B}$ ，使用状态映射函数简单地删除优化添加的新变量。为了证明 $\mathcal{B}^\Delta$ 改进了 $\mathcal{B}$ ，我们通过分析我们方法的三种情况来证明 $\mathcal{B}^\Delta$ 是非变异优化。我们将细节留给[37]。

5 将 PAXOS 优化移植到 RAFT*。

Paxos 变体和优化的前景。我们首先使用我们方法的视角来研究现有的 Paxos 变体和优化。图 6 显示了这些协议之间的关系。其中，六个协议属于 Paxos 的非变异优化类别，如图 6 中的双线框所示。因此，这六个优化可以使用我们的方法从 Paxos 移植到 Raft*。灵活的 Paxos [11] 放宽了 Paxos 中的多数仲裁限制，以允许不同大小的仲裁，只要保证 Paxos 交换的两个阶段中使用的仲裁相交即可。因此，Paxos 改进了 Flexible Paxos，但反之则不然。WPaxos [3] 是最近提出的针对 Flexible Paxos 的非变异优化。因此，我们的方法也可以用于将 WPaxos 的优化移植到 Paxos。

至于其余的协议变体（如图 6 最左侧的方框所示），它们与 Paxos 的关系无法通过细化映射来捕获。缺乏精细化映射的原因是多种多样的。例如，Fast Paxos 更改了 Paxos 的仲裁大小以包含绝对多数，这阻止了从 Fast Paxos 到 Paxos 的细化映射。然而，它也错过了 Paxos 中允许的状态转换，这妨碍了从 Paxos 到 Fast Paxos 的细化映射。

在六个候选协议中，我们选择移植两个优化作为案例研究：Paxos Quorum Lease [28] 和 Mencius [25]。我们解释这些优化是什么以及它们如何移植到 Raft*。为了简单起见，我们的讨论使用伪代码而不是 TLA⁺。更正式的描述（包括细化映射、伪代码和 TLA⁺）可以在论文 [37] 的扩展版本中找到。

Paxos 法定人数租赁。在 Paxos 中，强一致性读取操作是通过将操作持久化到日志中来执行的，就好像它是写入一样。Paxos Quorum Lease (PQL) [28] 引入了一种优化，如果副本持有来自副本仲裁 (quorum-lease) 的租约，则允许任何副本在本地读取。

Quorum-lease 可以与 Paxos 中的 quorum 完美共存。任何副本都可以授予租约。如果副本拥有来自法定数量副本的租约，则副本认为自己拥有法定数量租约。任何租约法定人数必须与任何 Paxos 法定人数重叠（通常两者都

法定人数是副本的大多数）。在 Paxos 中，任何提交都需要从确认法定人数中收集，这将与租赁法定人数相交。因此，只要我们要求 Paxos 仲裁中的每个副本在副本提交任何值之前通知其授予的租约持有者，系统就是安全的——读和写都是一致的。

PQL 是 Paxos 的非变异变体，因为它所有添加和修改的子操作都不会改变 Paxos 中的状态变量。图 7 显示了 PQL 引入的算法更改。更改的操作是 Phase2b 和 Learn。在 Phase2b 中，服务器附加它通过“acceptOk”响应授予的所有租约（第 143 行）。通过从仲裁中收集授予的租约（第 146 行），学习者可以找到所有持有有效租约的服务器（第 148 行），并且仅当它从这些持有者那里收到“acceptOk”时才提交实例（第 150 行）。添加的操作是 Read 和 LocalRead。当客户端发出“local Read”请求时，如果服务器拥有法定人数授予的有效租约，并且所有提交的更新都在所选集中，则服务器将返回其本地副本（第 136-137 行）。

筏*-PQL。图 8 显示了将 PQL 应用于 Raft* 后的算法。蓝色代码显示了将优化移植到 Raft* 后发生变化的部分。对于要执行本地读取的副本，副本需要检查两个条件是否成立。首先，副本必须持有至少 $f + 1$ 个副本（包括其自身）的租约。其次，副本需要等待，直到 `commitIndex` 大于修改目标记录的条目的最大索引（第 156 行）。这是从 PQL 转换而来的，其中所有修改都必须在 `selectedSet` 中（第 137 行）。

副本将自己授予的租约持有者附加到 `appendOk` 消息中，该消息映射到 `acceptOk` 消息。在 `LeaderLearn` 中，领导者需要收集消息中附加的、由其自身授予的租约的持有者（第 165 行）。这是因为带有一个额外隐式 `appendOk` 消息的 f 个 `appendOk` 消息意味着 Paxos 中的 $f + 1$ 个 `acceptOk` 消息。因此，收集 $f + 1$ 消息中附加的租约（第 148 行）被转换为收集来自 f 消息和本地授予的租约（隐式消息）。

孟子。Paxos 要求将所有客户端请求发送给领导者以获得更好的吞吐量。这可能会导致领导者副本和其他副本之间的负载不平衡。当副本位于不同的数据中心时，非领导者副本将需要至少两次广域往返才能提交任何请求，因为请求需要转发给领导者。为了解决这些问题，Mencius [25] 对 Paxos 实例进行分区，以便每个副本充当不同实例子集的默认领导者。通过地理副本，客户端可以将其请求发送到最近的副本。副本可以使用其作为默认领导者的那些 Paxos 实例提交这些请求。这样，孟子就可以平衡所有副本之间的负载，同时也减少了广域往返。

孟子以循环方式对实例（日志）空间进行分区。例如，在具有三个副本 r_1 、 r_2 、 r_3 的系统中， r_1 是日志条目 $(0, 3, 6, \dots)$ 的默认领导者， r_2 是 $(1, 4, 7, \dots)$ 的领导者， r_3 是 $(2, 5, 8, \dots)$ 的领导者。孟子将日志条目的执行与其提交分开。日志是按顺序执行的。如果某个默认领导者尚未收到任何用户提交的操作，它会提交跳过条目以保持日志执行继续进行。为了防止崩溃的副本延迟系统，属于该副本的实例可以由其他副本以无操作方式提交。

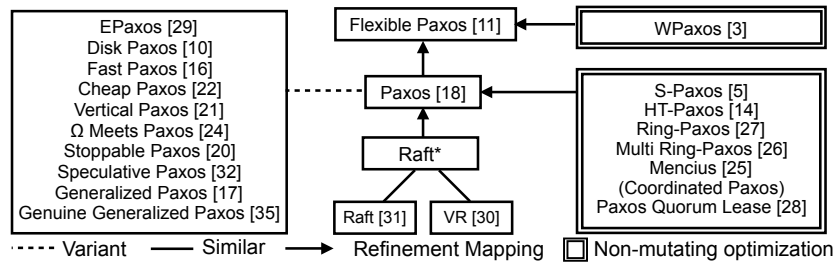


图6: 不同Paxos变体与Raft*的关系

化。

```

128 function Read(c, k):
129     c sends <'localRead', k> to 1 server s
130     if c receives <'ReadReply', v> from s
131     then
132         return v
133
134 function LocalRead(s):
135     if s receives <'localRead', k>
136         && s.validLeasesNum ≥ f + 1
137         && all instances modified k are in chosenSet
138     then
139         s replies <'ReadReply', LocalCopy(k)>
140
141 function Phase2b(s):
142     ...
143     s replies <'acceptOk', ..., leases granted by s>
144
145 function Learn(s):
146     if s receives <'acceptOk', i, v, b, s, leases> from f + 1
147         acceptors
148     then
149         holderSet = holders of received leases
150     if s receives <'acceptOk', ... > from all holders in holderSet
151     then
152         ...

```

图 7: Paxos Quorum 租赁

```

153 function LocalRead(s): 154 if s receive < 'localRead' , k> 155 && s.validLeasesNum ≥ f + 1 156 && s.log 中条目的索引修改 k ≤ s.commitIndex 157 then 158 ... 159 160 function LeaderLearn(s): 161如果 s 从 f 接受者接收到 < 'appendOK' , t, index, holder> 162 && s.isLeader 163 && s.currentTerm == t 164 那么 1 165holderSet = 接收到领导者授予的租约的持有者 ∪ 持有者 166 如果 s 从所有持有者那里接收到 < 'appendOK' , ... >固定器设置 167 然后 168 ...

```

图 8: Raft*-PQL

这些优化帮助 Mencius 平均在 1.5 次往返内提交和执行请求。

筏*-孟子。Raft*-Mencius 的完整伪代码包含在[37]中。我们在这里描述一些有趣的细节。除了 Paxos 状态变量之外，每个副本还需要保留一个数组

“skip-tags”来跟踪那些可以跳过的日志条目。当副本成为领导者时，它不仅需要收集其他副本的值，还需要收集跳过标签。因为Paxos中的Phase2b动作对应于Raft*中的许多动作（AppendEntries、ReceiveAppend），所以孟子对Phase2b所做的任何改变也应该应用于这些动作。例如，如果新附加的条目是跳过条目，则应将它们标记为可执行。

由于 Paxos 中的 Phase2b 隐含在 Raft* 中的多个子操作中，因此移植 Mencius 的手动工作可能会错过某些操作。例如，如果手动解决方案仅将 Paxos 中 Phase2b 上的更改应用于 Raft* 中的 ReceiveAppend 并错过 AppendEntries，则该解决方案可能会错过一些优化机会，甚至导致协议不正确。

6 评价

本节显示生成的算法与 Paxos 算法实现了类似的优化效果 [28, 25]。特别是，Raft*-PQL 通过执行本地读取来减少读取请求的延迟。Raft*-Mencius 通过平衡所有副本的工作负载来提高吞吐量。我们的移植方法保证了这些派生算法的正确性。但具体实施仍需付出额外努力。

试验台。这些实验是在 5 个不同地理区域的 Amazon EC2 上进行的：俄勒冈州、俄亥俄州、爱尔兰、加拿大和首尔。在每个区域中，两个 m4.xlarge 实例分别用于客户端和服务端进程。每个实例有 4 个虚拟 CPU、16GB 内存和 1 个 SSD，带宽为 750 Mbps。站点之间的延迟从 25 毫秒到 292 毫秒不等。

工作量。我们的评估使用具有类似 YCSB 工作负载的闭环客户端：每个客户端连续发出 get 或 put 请求。系统初始化时有 100K 条记录。为了模拟争用，每个客户端以配置的速率（即冲突率）访问相同的流行记录。当不访问流行记录时，密钥空间在数据中心之间均匀地预先分区，并以均匀的概率从该分区中选择密钥。默认情况下，Raft*-PQL 以 90% 的读取率进行评估。对于 Raft*-Mencius，我们使用 100% 写入的工作负载。每个试验运行 50 秒，其中热身和放松各 10 秒。报告的每个数字都是 5 次试验的中位数。

执行。Raft* 的实现基于流行的工业 Raft 代码库 —etcd（版本 c4fc8c09）。Etcd 有一些重要的优化。首先，它具有定制的网络层以实现高效通信。其次，当追随者收到来自客户端的多个请求时，它将它们转发给领导者

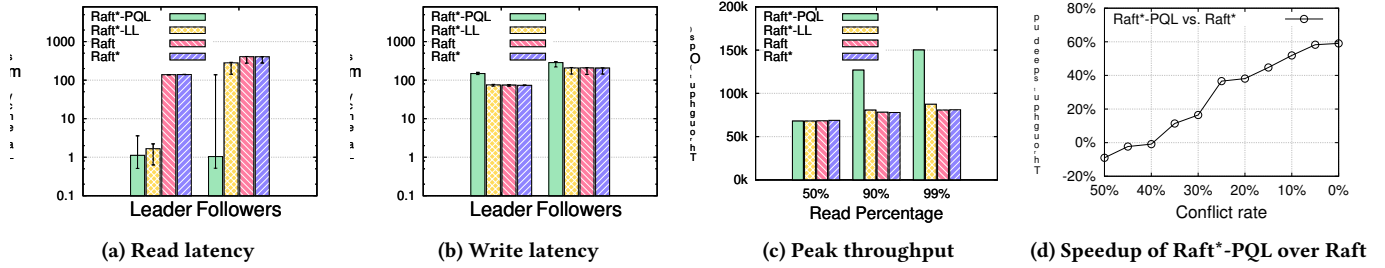


图 9: Raft*-PQL 与 LL 与 Raft。 (a) 和 (b) 中的每个条代表带有错误条 fr 的请求的第 90 个百分位延迟第 50 到 99 个百分位数。 (a) 和 (b) 的 y 轴采用对数刻度。

奥姆

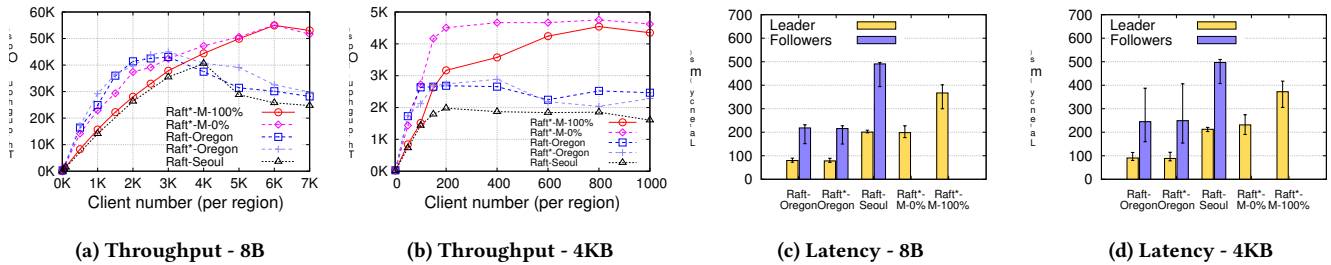


图 10: Raft*-孟子与 Raft。 Raft*-M-100%和Raft*-M-0%代表Raft*-Mencius, 工作负载低于100%和0%冲突率。 Raft-Oregon 和 Raft-Seoul 代表领导者是在俄勒冈州还是在首尔。 8B和4KB是工作负载的请求大小。

批。当跟随者服务器接受客户端请求时, 此类技术可以提高系统吞吐量。在我们的测试中, 打开这些优化后, etcd 的吞吐量提高了 2.4x。我们在测试中保留这些优化, 以赋予 etcd 额外的优势。俄勒冈州被用作 etcd 的领导站点, 这给了它最好的结果, 因为俄勒冈州拥有最好的网络条件。

6.1 筏*-PQL

我们使用[28]中相同的租约参数评估 Raft*-PQL: 租约持续时间为 2 秒, 每 0.5 秒更新一次。我们还将 Raft*-PQL 与 Leader Lease (LL) 进行比较。这里领导者拥有租约的唯一所有权, 因此只有领导者可以使用其本地副本处理读取请求。默认情况下, 我们使用 90% 的读取工作负载和 5% 的冲突率。

延迟。首先, 我们比较每个区域 50 个客户端的延迟。在 Raft*-PQL 中, 任何具有有效租约的服务器都能够进行本地一致读取, 因此 90% 的读取请求只有 1ms 的延迟 (图 9a)。相比之下, 对于 LL, 只有领导者可以处理类似延迟 (1.6 ms) 的读取请求。Raft*-PQL 有 1% 的读取请求在关注者上具有高延迟 (~137ms)。这是因为工作负载中存在 5% 的争用: 追随者需要等待冲突的写入提交才能处理读取请求。对于写入延迟 (图 9b), Raft*-PQL 比其他系统稍高一些, 因为它需要等待租约持有者确认才能提交写操作, 而其他系统总是可以选择最快的多数。

吞吐量。图 9c 显示了读取百分比 (50%、90% 和 99%) 对峰值吞吐量的影响。筏、筏* 和

LL 实现了几乎相同的峰值吞吐量, 因为领导者的 CPU 是瓶颈, 并且饱和的领导者 CPU 具有相同的能力来处理这些算法的请求。相比之下, Raft*-PQL 以 90% 和 99% 的读取速度实现 1.6x 和 1.9x 加速。优点是在本地执行读取请求。图 9d 显示了冲突率如何影响吞吐量加速。该图没有显示 Raft*-PQL 相对于 Raft* 的加速, 因为它们是相似的。加速随着冲突率的降低而增加, 因为所有跟随者都可以在低冲突率下立即将读取请求返回给用户。

6.2 筏*-孟子

我们使用 100% put 工作负载来衡量具有 0% 和 100% 竞争的 Raft*-Mencius, 分别标记为 Raft*-Mencius-0% 和 Raft*-Mencius-100%。为了进行公平比较, 我们通过将领导者放置在距所有其他区域 (Raft-Oregon 和 Raft-Seoul) 最近 (俄勒冈) 和最远 (首尔) 服务器中来评估 Raft 在广域中的最佳和最差情况。我们仅与俄勒冈州的领导者一起评估 Raft* 以供参考。

吞吐量。图 10a 给出了系统受 CPU 限制时的吞吐量。Raft*-Mencius 可以实现每秒 55K 操作 (ops) 的峰值吞吐量, 因为它平衡了所有副本之间的负载。相比之下, 其他系统在其领导者的 CPU 饱和后可以达到 41K 操作的峰值吞吐量。图 10b 给出了系统受网络限制时的吞吐量。Raft 在领导者的网络带宽饱和后达到峰值吞吐量。Raft-Oregon 的吞吐量比 Raft-Seoul 高 30%

因为俄勒冈州有更高的带宽。Raft*-Mencius 的吞吐量比 Raft-Oregon 高 70%，因为它能够利用所有副本的网络带宽。在这两个图中，在客户端数量较少的情况下，Raft-Oregon 和 Raft*-Mencius-0% 由于其较低的延迟而比其他客户端具有更好的性能。

延迟。图 10c 和图 10d 显示了每个区域 50 个客户端的延迟。Raft-Oregon 的领导者以最低延迟（79ms）处理请求，因为俄勒冈州、俄亥俄州和加拿大的法定人数最接近。相比之下，Raft*-Mencius- 100% 的 90% 百分位延迟要高得多，而 Raft*-Mencius- 0% 由于竞争级别不同而具有较低的延迟。

7 相关工作

基本共识协议。除了 Raft [31] 和 Paxos [18] 之外，还有很多替代协议。例如，Viewstamped Replication (VR) [30] 的发布早于 Paxos，ZooKeeper [12] 使用 ZAB [13]。我们的方法也适用于这些协议。特别是，我们可以通过制作类似 Raft* 的协议来将这些协议与 Paxos 连接起来。

算法比较。雷内塞等人。[36]使用细化映射将 Paxos 与 VR 和 ZAB 进行比较。Lamport[15]讨论了拜占庭Paxos和PBFT[7]之间的等价性。宋等人。[34] 确定了 Paxos、Chandra-Toueg [8] 和 Ben-Or [4] 共识算法的共同特征。Abraham 和 Malkhi [2] 讨论了 BFT 共识协议和区块链协议之间的联系。与这些工作相比，我们有两个显着的区别：我们使用形式化方法 TLA^+ [19] 来建模细化映射 [1]；我们已经机械地将优化从一个协议系列导出到另一个协议系列。

Paxos 变体和优化。图 6 显示了许多 Paxos 变体。在非变异变体中，WPaxos [3] 对对象进行分区并使用灵活的仲裁进行异地复制 [11]；HT-Paxos [14] 和 S-Paxos [5] 将排序任务分配给多个服务器以消除瓶颈。Ring Paxos [27]和Multi RingPaxos [26]对工作负载进行分区并获得更好的性能。在不断变化的 Paxos 变体中：Cheap Paxos [22] 引入了辅助服务器。 Ω 遇见 Paxos [24] 在弱网络环境中选出稳定的领导者。NetPaxos [9]使Paxos适应SDN。Stoppable Paxos [20] 能够在不减慢速度的情况下执行重新配置。此外，Shraer 等人。[33]和Vertical Paxos [21]讨论了如何重新配置复制状态机。Disk Paxos [10] 在磁盘集群中达成共识。Fast Paxos [16]和多协调Paxos [6]引入了快速仲裁来通过单次往返达成共识。Generalized Paxos [17]、Genuine Generalized Paxos [35] 和 EPaxos [29] 解决了执行时的冲突。推测性 Paxos [32] 在消息按顺序传递时引入推测执行。

致谢

我们衷心感谢匿名审稿人富有洞察力的评论。我们感谢 Lamon t Nelson 的帮助，感谢 Aurojit Panda 对这项工作的反馈。这项工作得到了中国国家重点研发计划（编号：2016YFB1000104）、NSF 拨款 CNS-1409942 和 CNS-1514422 以及 AFOSR FA 9550-15-1-0302 的支持。

参考

- [1] M. Abadi 和 L. Lamport 细化映射的存在。理论计算机科学 82, 2 (1991).
- [2] I. Abraham, D. Malkhi, K. Nayak, L. Ren 和 A. Spiegelman Solida: 一种基于可重构拜占庭共识的加密货币。在过程中。奥波迪斯。2017.
- [3] A. Aili Jiang, A. Charapko, M. Demirbas 和 T. Kosar WPaxos: 广域网灵活共识。arXiv 预印本 arXiv:1703.08905 (2017).
- [4] M. Ben-Or 自由选择的另一个优点（扩展摘要）：完全异步的协议协议。在过程中。PODC。1983 年 8 月。
- [5] M. Biely, Z. Milosevic, N. Santos 和 A. Schiper S-Paxos: 卸载高吞吐量状态机复制的领导者。在过程中。SRDS。2012 年 10 月。
- [6] L. J. Camargos, R. M. Schmidt 和 F. Pedone 多协调 Paxos。在过程中。PODC。2007 年 8 月。
- [7] M. Castro 和 B. Liskov 实用拜占庭容错。在过程中。OSDI。1999 年 2 月。
- [8] T. D. Chandra 和 S. Toueg 用于可靠分布式系统的不可靠故障检测器。JACM 43, 2 (1996 年 3 月)。
- [9] H. T. Dang, D. Sciascia, M. Canini, F. Pedone 和 R. Soulé Netpaxos: 网络速度的共识。第一届 ACM SIGCOMM 软件定义网络研究研讨会论文集。2015.
- [10] E. Gafni 和 L. Lamport 磁盘 paxos。直流电 (2003)。
- [11] H. Howard, D. Malkhi 和 A. Spiegelman 灵活的 paxos: 重新审视 Quorum 交叉。arXiv 预印本 arXiv:1608.06696 (2016)。
- [12] P. Hunt, M. Konar, F. P. Junqueira 和 B. Reed ZooKeeper: 互联网规模系统的无等待协调。在过程中。USENIX ATC。2010 年 6 月。
- [13] F. P. Junqueira, B. C. Reed 和 M. Serafini Zab: 主备份系统的高性能广播。在过程中。DSN。2011 年 6 月。
- [14] V. Kumar 和 A. Agarwal HT-Paxos: 大型集群数据中心的高吞吐量状态机复制协议。科学世界杂志 (2015)。
- [15] L. Lamport 通过细化拜占庭化 Paxos。在过程中。光盘。2011 年 7 月。
- [16] L. Lamport Fast Paxos。DC 19, 2 (2006 年 10 月)。
- [17] L. Lamport 广义共识和 Paxos。技术。代表。MSR-TR-2005-33。Microsoft Research, 2005.
- [18] L. Lamport Paxos 变得简单。SIGACT 32, 4 (2001)。
- [19] L. Lamport 指定系统：面向硬件和软件工程师的 TLA^+ 语言和工具。Addison-Wesley Longman Publishing Co., Inc., 2002.
- [20] L. Lamport, D. Malkhi 和 L. Zhou 重新配置状态机。ACM SIGACT 新闻 (2010)。
- [21] L. Lamport, D. Malkhi 和 L. Zhou 垂直 Paxos 和主备复制。在过程中。PODC。2009 年 8 月。
- [22] L. Lamport 和 M. Massa Cheap Paxos。在过程中。DSN。2004 年 6 月。
- [23] L. Lamport 和 S. Merz TLA^+ 中的辅助变量。arXiv 预印本 arXiv:1703.05121 (2017)。
- [24] D. Malkhi, F. Oprea 和 L. Zhou Ω 遇见 paxos: 没有最终及时联系的领导者选举和稳定性。在过程中。光盘。2005.
- [25] Y. Mao, F. P. Junqueira 和 K. Marzullo Mencius: 为 WAN 构建高效的复制状态机。在过程中。OSDI。2008 年 12 月。
- [26] P. J. Marandi, M. Primi 和 F. Pedone 多环 paxos。在过程中。DSN。2012.
- [27] P. J. Marandi, M. Primi, N. Schiper 和 F. Pedone Ring Paxos: 一种高吞吐量原子广播协议。在过程中。DSN。2010.
- [28] I. Moraru, D. G. Andersen 和 M. Kaminsky Paxos 仲裁租约：快速读取而不牺牲写入。在过程中。SoC。2014 年 11 月。
- [29] I. Moraru, D. G. Andersen 和 M. Kaminsky 在平等主义议会中有更多共识。在过程中。SOSP。2013 年 11 月。
- [30] B. M. Oki 和 B. H. Liskov Viewstamped 复制：一种支持高可用性分布式系统的新主复制方法。在过程中。PODC。1988 年 6 月。
- [31] D. Ongaro 和 J. K. Ousterhout 寻找一种可理解的共识算法。在过程中。USENIX ATC。2014 年 6 月。
- [32] D. R. Ports, J. Li, V. Liu, N. K. Sharma 和 A. Krishnamurthy 在数据中心网络中使用近似同步设计分布式系统。在过程中。NSDI。2015 年 5 月。
- [33] A. Shraer, B. Reed, D. Malkhi 和 F. P. Junqueira 主/备份集群的动态重新配置。在过程中。USENIX ATC。2012 年 6 月。
- [34] Y. J. Song, R. van Renesse, F. B. Schneider 和 D. Dolev 共识的基石。在 IEEE ICDCN 中。2008 年 1 月。
- [35] P. Sutra 和 M. Shapiro Fast 真正的普遍共识。在过程中。SRDS。2011 年 10 月。
- [36] R. Van Renesse, N. Schiper 和 F. B. Schneider Vive la différence: Paxos, viewstamped 复制与 zab。IEEE 可靠和安全计算汇刊 12, 4 (2015 年 7 月)。
- [37] W. 赵国, Z. 长庚, M. 帅, C. 海波, L. 金阳谈 Paxos 和 Raft 的相似之处以及如何移植优化（扩展版）。arXiv 预印本 arXiv:1905.10786 (2019)。